

UNIVERSIDAD NACIONAL EXPERIMENTAL DEL TÁCHIRA

VICERRECTORADO ACADÉMICO

DECANATO DE DOCENCIA

DEPARTAMENTO DE ING. EN INFORMÁTICA

UNIDAD CURRICULAR:

COMPUTACIÓN I

(Código: 0415102T)

Clase No. 2

Unidad II: Algoritmos y Programación Estructurada

Profesor: Armando Carrero

Unidad Tema .

II Algoritmos y Programación Estructurada

Pasos para Resolver un Problema usando el Computador (*Metodología*):

1.- Definición e Interpretación del problema.

(Implica Leer, comprender e interpretar)

2.- Análisis del problema.

(Determinar: Datos de Entrada, Información de Salida, Restricciones o condiciones)

3.- Planificación de la Solución.

(Generar una solución paso a paso. Construir un ALGORITMO)

4.- Prueba de Escritorio

(Verificar la solución usando datos ficticios o reales para los que se conozcan los resultados. Se conoce como: Prueba de escritorio o Corrida en frio)

5.- Codificación del Algoritmo

(Generar el programa fuente usando la sintaxis de un lenguaje de programación)

6.- Transcripción y Compilación

(Editar el programa en el Computador y generar el programa objeto)

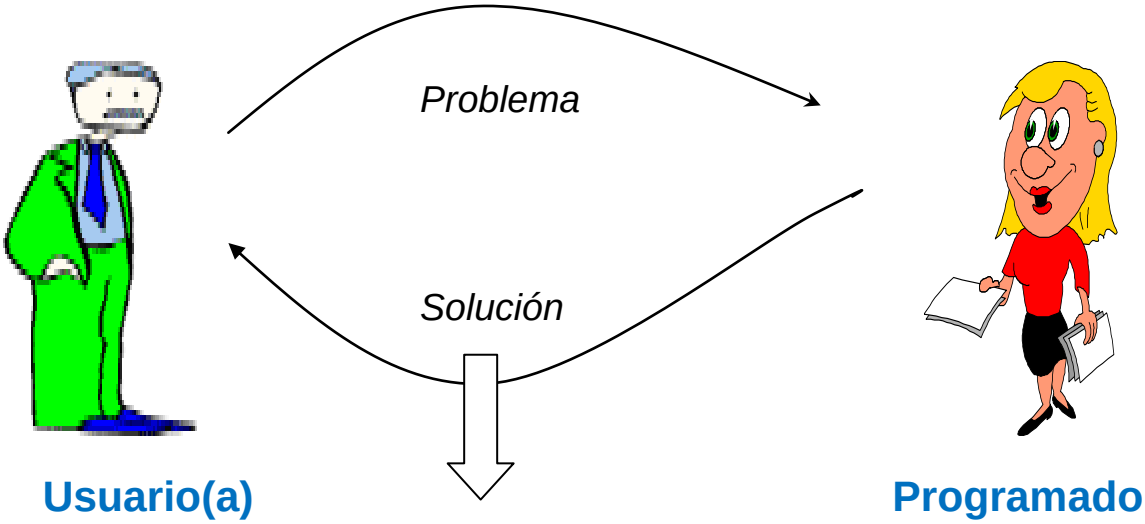
7.- Prueba y Depuración

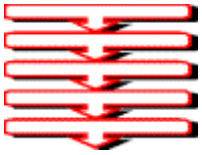
(Probar el programa usando datos ficticios y corregir errores de sintaxis o errores lógicos)

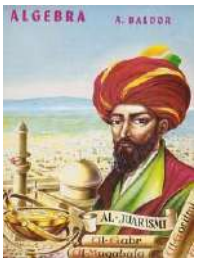
8.- Documentación y Mantenimiento

(Generar manual de operaciones y realizar revisión, actualización o modificación del programa)

Planteamiento del problema, por parte de un usuario(a), y del trabajo a realizar por el programador(a).







Nombre de Algoritmo: Se debe en honor al matemático árabe, astrónomo, geógrafo, del siglo IX:

Muhammad Musa al-Juarismi

¿Qué es un Algoritmo?: Es una secuencia de pasos ordenados lógicamente, necesarios para llevar a cabo una tarea específica.

Un Algoritmo se diseña usando lo que se denomina: Seudocódigo o Seudolenguaje (*forma de expresar los distintos pasos que va a realizar un programa, de la forma más parecida a un lenguaje de programación*).

Características de un Algoritmo:

1.- Debe ser Finito (matemáticamente, debe tener Principio y Fin)
2.- Debe ser Preciso (Exacto, sin ambigüedades)
3.- Debe ser general (Universal)
4.- Debe generar al menos una salida (Información).
5.- Es posible que no requiera de datos de entrada (Forma externa)
6.- Debe ser optimo (Mínimo uso de recursos)

Un algoritmo se alimenta de datos, para producir resultados:



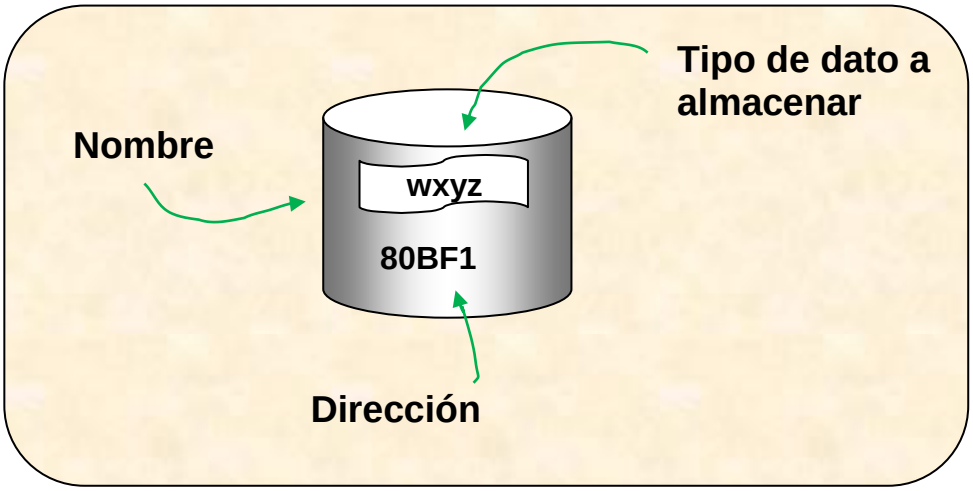
Los datos, valores y los resultados deben almacenarse en un lugar denominado:

Variable

Concepto de Variable: Lugar de almacenamiento de Valores (Datos, Valores intermedios o Resultados)

Características de una Variable:

Una variable posee tres características:



Nombre: Lo asigna el programador y aunque no hay reglas para denominarlo, se recomienda que esté asociado al valor que almacenará, usando en ocasiones abreviaturas para evitar nombres largos. Ejemplos:

Nombre	Valor a almacenar
cédula	Identificación de una persona
sueldo	Sueldo de un empleado
tempC	Temperatura en grados centígrados
intelec	Intensidad eléctrica en amperios

Tipo de Datos:

Tipo		Valor(Ejemplo)	Para almacenar (Ejemplo)
ENTERO	(Números Enteros)	18	Edad de un alumno
		10	Cantidad de autos
		15	Nota definitiva
REAL	(Números Reales)	14,50	Altura de una torre
		117,75	Presión promedio
CADENA	(Cadenas de Caracteres)	"UNET"	Nombre de Institución
		"baloncesto"	Deporte
		"X-000001"	Código
CARÁCTER	(Único carácter)	'F'	Sexo
		'A'	Sección
		'E'	Nacionalidad
LOGICO	(Valor Lógico)	FALSO o VERDADERO	Una Clave ó Resultado de una comparación

Dirección: La asigna el software instalado en la computadora, al ejecutar el programa, es un valor en el sistema hexadecimal y sirve para acceder al valor de la variable mediante herramientas de programación, que se estudiaran en la unidad VI.
Ejemplo de una dirección de una variable: 0022FF1C

Partes de un Algoritmo:

```
❶ INICIO   Algoritmo Cálculo del Indice Académico (UNET)
❷ DEFINIR  Entero    x , a , b , nota , unidad
           Real      índice
           Cadena    nom
❸ a ←— 0 , b ←— 0

  ESCRIBIR " Ingrese el Nombre del alumno"
  LEER nom

  HAGA DESDE  x ←— 1   HASTA  7
     Comienzo
        ESCRIBIR " Ingrese Nota de la asignatura No." , x , "y su valor en
                   unidades credito"
        LEER  nota, unidad
        a ←— a  +  nota . unidad
        b ←— b  +  unidad
     Fin el Haga Desde
  indice ←—  a ÷ b
  ESCRIBIR  nom, " obtuvo un indice académico de: ", indice
  SI  indice <  5,10
     ENTONCES
           ESCRIBIR " Alumno en periodo de prueba "
     CASO CONTRARIO
           ESCRIBIR " Alumno en situación Regular"
     FIN DE SI
  FIN del Algoritmo Indice Academico
```

Identificación de las partes:

1.- **Encabezamiento**

2.- **Sección de Declaraciones**

3.- **Cuerpo Principal**

Herramientas o Enunciados para Diseñar Algoritmos

1.- Para Iniciar todo algoritmo:

formato:	**INICIO**	**Nombre del Algoritmo**
ejemplo:	**INICIO**	**Algoritmo Indice Academico**

2.- Para finalizar todo algoritmo:

formato:	**FIN**	**Nombre del Algoritmo**
ó	**FIN** .	
ejemplo:	**FIN**	**Algoritmo Indice Academico**

3.- Para Declarar las variables:

formato:	**DEFINIR**	**tipo de dato**	**Nombre(s) de Variable(s)**
ejemplo:	**DEFINIR**	**Entero**	**x , cantidad**
		Cadena	**código**
		Real	**peso, sueldo**
		Carácter	**h**
		Logico	**clave**

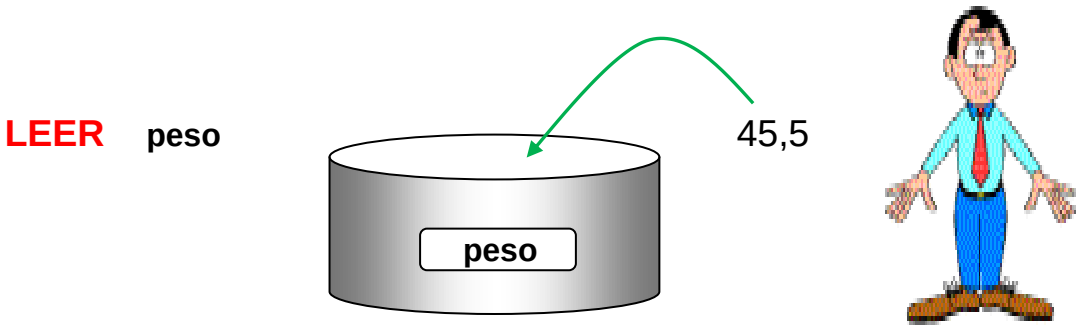
4.- Para ingresar y almacenar los datos de entrada:

Tipos de Entrada :
- Forma Interna (*al algoritmo*): El algoritmo se los autosuministra.
- Forma Externa (*al algoritmo*): Los suministra el usuario.

Para dar entrada a los datos en **forma externa**, por parte del usuario:

formato:	**LEER**	**Nombre de Variable (s)**
ejemplo:	**LEER**	**x**
	LEER	**código**
	LEER	**peso, sueldo**

Al ejecutar ésta instrucción, sucede que el proceso se detiene para que el usuario, ingrese el valor correspondiente y éste se almacene en la variable indicada. Asi, consideremos la instrucción siguiente:



Al encontrarse la instrucción (LEER peso), la ejecución del algoritmo se detiene, a la espera que el usuario le de entrada a un valor. Luego de la entrada del dato (Ejemplo: 45,5), la variable (peso) contendra dicho valor (45,5) y cada vez que en el algoritmo se procese la variable “peso”, se hara referencia a su valor.

Nota: Una variable(Simple), solo puede almacenar un valor a la vez; en caso de almacenar un nuevo valor, el anterior se destruye, es decir es sustituido por el nuevo valor. En ésta parte del contenido, Unidad II, se trabajara con variables de tipo simple y posteriormentre en la Unidad IV y VII se manejaran otro tipo de variables.

Para dar entrada a los datos en **forma interna**:

formato: **Nombre de Variable** ← **Valor, Variable o Expresión**

A esta forma, también se le denomina sentencia de asignación, entendiéndose que el valor de la derecha, o el valor almacenado en la variable de la derecha, o el resultado de la expresión de la derecha, se asigna a la variable ubicada en la izquierda.

Ejemplos, tomando la declaración de variables indicada anteriormente:

```
DEFINIR   Entero      x , cantidad
          Cadena      código
          Real        peso, sueldo
          Carácter    h
          Logico      clave
```

Asignación de un valor:

Ejemplos				
	Entero	**cantidad**	←	**150**
	Cadena	**código**	←	**“X-12501”**
	Real	**sueldo**	←	**150,75**
	Carácter	**h**	←	**‘ C ‘**
	Logico	**clave**	←	**VERDADERO**

Nota: Los valores tipo cadena deben encerrarse entre comillas dobles.
Los valores tipo caracter deben encerrarse entre comillas simples.

Asignación de una variable a otra variable:

Ejemplo **x** ← **cantidad**

Se entiende que el valor de la variable cantidad, se guarda ahora en la variable x, es decir, si previamente a la variable cantidad se le asignó el valor 150, ahora x también contiene ese valor.

Asignación de una expresión:

TIPOS DE EXPRESIONES:
- **Aritméticas**
- **Relacionales**
- **Lógicas**

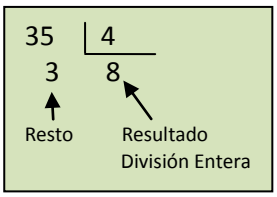
EXPRESIONES ARITMETICAS: Son aquellas expresiones matemáticas, en donde los términos están relacionados o concatenados mediante operadores aritméticos.

OPERADOR ARITMETICO		OPERACIÓN
+	.....	Suma
-	.....	Resta
• ó x	.....	Multiplicación
÷ ó — ó /	.....	División
( ) ^{n}	.....	Potencia enésima
Log ( )	.....	Logaritmo decimal
Ln ( )	.....	Logaritmo neperiano
$\sqrt[n]{(\ )}$	.....	Raíz enésima
⟦ ⟧	.....	Parte entera
rst (a ÷ b)	.....	Resto de División entera
Otros.....		

Ejemplos:

d	←	$b^2 - 4.a.c$
v	←	e / t
m	←	$\frac{y2-y1}{x2-x1}$
ºf	←	$\frac{9}{5}$ °C – 32
c	←	$\sqrt{a^2 + b^2}$
x	←	$\frac{-b+\sqrt{b^2-4ac}}{2a}$

a ←	2	Se asigna 2 a la variable a
b ←	5,5	Se asigna 5,5 a la variable b
c ←	a^2	Se asigna 4 a la variable c
d ←	35 ÷ c	Se asigna 8,75 a la variable d (equivale a: d ← 35 / c) (*División Real*)
e ←	$[\![35 \div c]\!]$	Se asigna 8 a la variable e (*División Entera*)
f ←	rst (35 ÷ c)	Se asigna 3 a la variable f
g ←	a + 25 / 2 - b	Se asigna 9,0 a la variable g



Nota: El resultado de una expresión numérica, es un valor numérico.

EXPRESIONES RELACIONALES: Son aquellas expresiones matemáticas, en donde los términos están relacionados o concatenados mediante operadores relacionales.

OPERADOR RELACIONAL	OPERACIÓN
>	mayor que
<	menor que
≤	menor o igual que
≥	mayor o igual que
=	igual que
≠ ó <>	diferente a .

Ejemplos:

n ←	10, 5	Se asigna a **n** el valor real 10,5
p ←	100 > n	Se asigna a **p** el valor logico VERDADERO, al comparar 100 > 10,5
q ←	π < 3	Se asigna a **q** el valor logico FALSO, , al comparar 3,14159 < 3
r ←	‘e‘ ≤ ‘E‘	Se asigna a **r** el valor logico FALSO. (Se comparan sus valores del codigo ASCII. Para ‘e’ es 101 y para ‘E’ es 69)
s ←	“papel” ≥ “casa”	Se asigna a **s** el valor logico VERDADERO. (Se comparan los valores, del codigo ASCII, del primer carácter de ambas cadenas, en caso de no haber diferencia, se continuan comparando los segundos caracteres, terceros y asi sucesivamente.[‘p’ (112) y ‘c’ (99)]
t ←	q < VERDADERO	Se asigna a **t** el valor logico VERDADERO. (Se asume que el valor VERDADERO es mayor al valor FALSO)
w ←	n ≠ 10,5	Se asigna a **w** el valor logico FALSO

Nota: El resultado de una expresión relacional, es un valor lógico.

EXPRESIONES LOGICAS: Son aquellas expresiones matemáticas, en donde los términos están relacionados o concatenados mediante operadores lógicos.

OPERADOR LOGICO	OPERACIÓN
^ (í lógico)	conjunción
v (ó lógico)	disyunción
~ ó ¬	negación

Conjunción: La conjunción de dos proposiciones (operandos), es cierta (VERDADERO) si y solo si, ambas proposiciones son ciertas. Las demás combinaciones resultan FALSO.

Considerando que los operandos deben poseer sólo valores lógicos, entonces, surgen únicamente, cuatro posibles combinaciones, mostradas en las conocidas “tablas de la verdad”.

Consideremos la expresión: A ← B ^ C (B y C son operandos con valores lógicos)

B	C	Valor asignado a **A**
FALSO	FALSO	FALSO
FALSO	VERDADERO	FALSO
VERDADERO	FALSO	FALSO
VERDADERO	VERDADERO	VERDADERO

Disyunción: La disyunción de dos proposiciones (operandos), es cierta (VERDADERO) cuando al menos una de ellas lo es. Así, el resultado será FALSO solo cuando ambos operandos lo sean.

Considerando que los operandos deben poseer sólo valores lógicos, entonces surgen, únicamente, cuatro posibles combinaciones, mostradas en las conocidas “tablas de la verdad”.

Consideremos la expresión: A ← B **v** C (B y C son operandos con valores lógicos)

B	C	Valor asignado a **A**
FALSO	FALSO	FALSO
FALSO	VERDADERO	VERDADERO
VERDADERO	FALSO	VERDADERO
VERDADERO	VERDADERO	VERDADERO

Negación: La negación de una proposición (operando) es el valor lógico contrario, es decir, será cierta cuando la proposición sea falsa y falsa cuando ésta sea falsa. Esta operación se manifiesta, también, en las conocidas "tablas de la verdad".

Consideremos la expresión: A ← ~ B (B es un operando con valor lógico)

B	Valor asignado a **A**
FALSO	VERDADERO
VERDADERO	FALSO

Ejemplos:

h	←	1000	Se asigna a **h** el valor entero 1000
i	←	$(h > 100) \wedge (h \neq 0)$	Se asigna a **i** el valor logico VERDADERO
j	←	$(\pi < 3) \vee (\text{'b'} > \text{'c'})$	Se asigna a **j** el valor logico FALSO
k	←	$(\text{' e'} \leq \text{' E '}) \wedge (h = 1000)$	Se asigna a **k** el valor logico FALSO
l	←	("Papel"<"Pastel") v (~i)	Se asigna a **l** el valor logico VERDADERO
q	←	~ j ^ ~ k	Se asigna a **q** el valor logico VERDADERO

Nota: El resultado de una expresión lógica, es un valor lógico.

¿Qué valor se asigna en las siguientes expresiones, a las variables que están a la izquierda en cada asignación?

1) X ← 12 + 8 ÷ 2

2) R ← FALSO ^ ~ FALSO

3) S ← rst (- 10 ÷ (5 + 2)) · 6

4) Q ← R v (S = - 18)

5) P ← Q v ~ Q ^ R <> R

6) Total ← 128 – rst (25 ÷ 9) x 4 + ⟦–17 ÷ 6⟧ - 3 ÷ 4

Se observa que se usan operadores de los tres tipos(aritméticos, relacionales y lógicos), luego para responder a la pregunta anterior, se debe atender a la jerarquia de operadores, que se muestra a continuación.

JERARQUÍA DE OPERADORES

Orden de Jerarquía	Símbolo/ Operador	Operación
1	( )	
2	~ ó ¬	Negación
3	x, ÷, rst, ⟦ ⟧	Multiplicación, División,Resto de división entera, Parte Entera
4	+, -	Suma, Resta
5	>, ≥, <, ≤	Mayor que, Mayor o igual que, Menor que, Menor o igual que
6	=, ≠ ó <>	Igual que, Diferente que
7	^	Conjunción (í lógico)
8	v	Disyunción (ó lógico)

Observacion:

1.- En aquellos casos en donde hallan operadores con el mismo nivel de jerarquía, se efectuará aquel que se encuentre primero de izquierda a derecha, en la expresión.

2.- El símbolo ( ), no es un operador, es un sigo de agrupación, que permite darle mayor jerarquía a un operador de mas bajo nivel, respecto a otro.

Solución a los Ejerccios Propuestos :

1) X ← 12 + 8 ÷ 2

Según la jerarquia:

.- primero se divide 8 ÷ 2 resultado parcial = 4

.- luego se suma 12 + 4

Resultado : Se asigna a la variable X, el valor **16**

2) R ⟵ FALSO ^ ~ FALSO

Según la jerarquia:

.- primero se niega el Falso (~ FALSO) resultado parcial = Verdadero

.- luego se realiza la conjunción (Falso ^ Verdadero) , según la tabla de la verdad.

Resultado : Se asigna a la variable R, el valor **FALSO.**

3) S ⟵ rst (- 10 ÷ (5 + 2)) · 6

Según la jerarquia :

.- primero se resuelve la suma 5 + 2 resultado parcial = 7

.- luego se halla el resto de dividir, en forma entera (sin extraer decimales), - 10 ÷ 7

```
-10 | 7      y se toma el resto        resultando parcial = - 3
+ 7 | -1
----
 -3
```

.- y finalmente se multiplica – 3 por 6 = - 18.

Resultado : Se asigna a la variable S, el valor **- 18**

4) Q ⟵ R v (S = - 18)

Según la jerarquia :

.- primero se compara el valor de la variable S, asignado en el ejercicio anterior No. 3, que resulto -18, con – 18 : (- 18 = - 18), dando como resultado Verdadero puesto que ambos valores son iguales:

resultado parcial = Verdadero

.- luego de determina la disyunción (v) con el valor de la variable R, asignado en el ejercicio No. 2, que resulto FALSO) y el resultado de la comparación anterior (Falso v Verdadero), luego según tabla de la verdad el resultado es Verdadero.

Resultado : Se asigna a la variable Q, el valor **VERDADERO**

5) P ⟵ Q v ~ Q ^ R < > R

Según la jerarquia:

.- primero se niega el valor de la variable Q, asignado en el ejercicio anterior No. 4, que resulto VERDADERO, (~ Verdadero), resultando Falso,
resultado parcial $_1$ = FALSO

.- luego se compara el valor asignado a la variable R, en el ejercicio anterior No. 2, que resulto Falso, (R < > R) es decir (Falso < > Falso) resultando Falso,
resultado parcial $_2$ = FALSO

quedando la expresion asi: Verdadero v Falso ^ Falso

.- luego, se determina la conjunción (^): Falso ^ Falso, resultando según la tabla de la verdad, Falso
resultado parcial $_3$ = FALSO

.- finalmente la disyunción (v): Verdadero v Falso, que según la tabla de la verdad resulta Verdadero.

Resultado : Se asigna a la variable P, el valor **VERDADERO**

6) Total ⟵ 128 – rst (25 ÷ 9) . 4 + ⟦−17 ÷ 6⟧ - 3 ÷ 4

Según la jerarquia :

.- primero se determina el valor de la operación resto de la division entera
rst (25 ÷ 9) resultado parcial $_1$ = 7

.- luego la operación parte entera
⟦−17 ÷ 6⟧ resultado parcial $_2$ = - 2

.- luego la operación división
3 ÷ 4 resultado parcial $_3$ = 0,75

.- quedando la expresión asi:
128 - 7 . 4 + (- 2) – 0,75

.- luego la operación multplicación
7 . 4 resultado parcial $_4$ = 28

.- luego la operación resta
128 - 28 resultado parcial $_5$ = 100

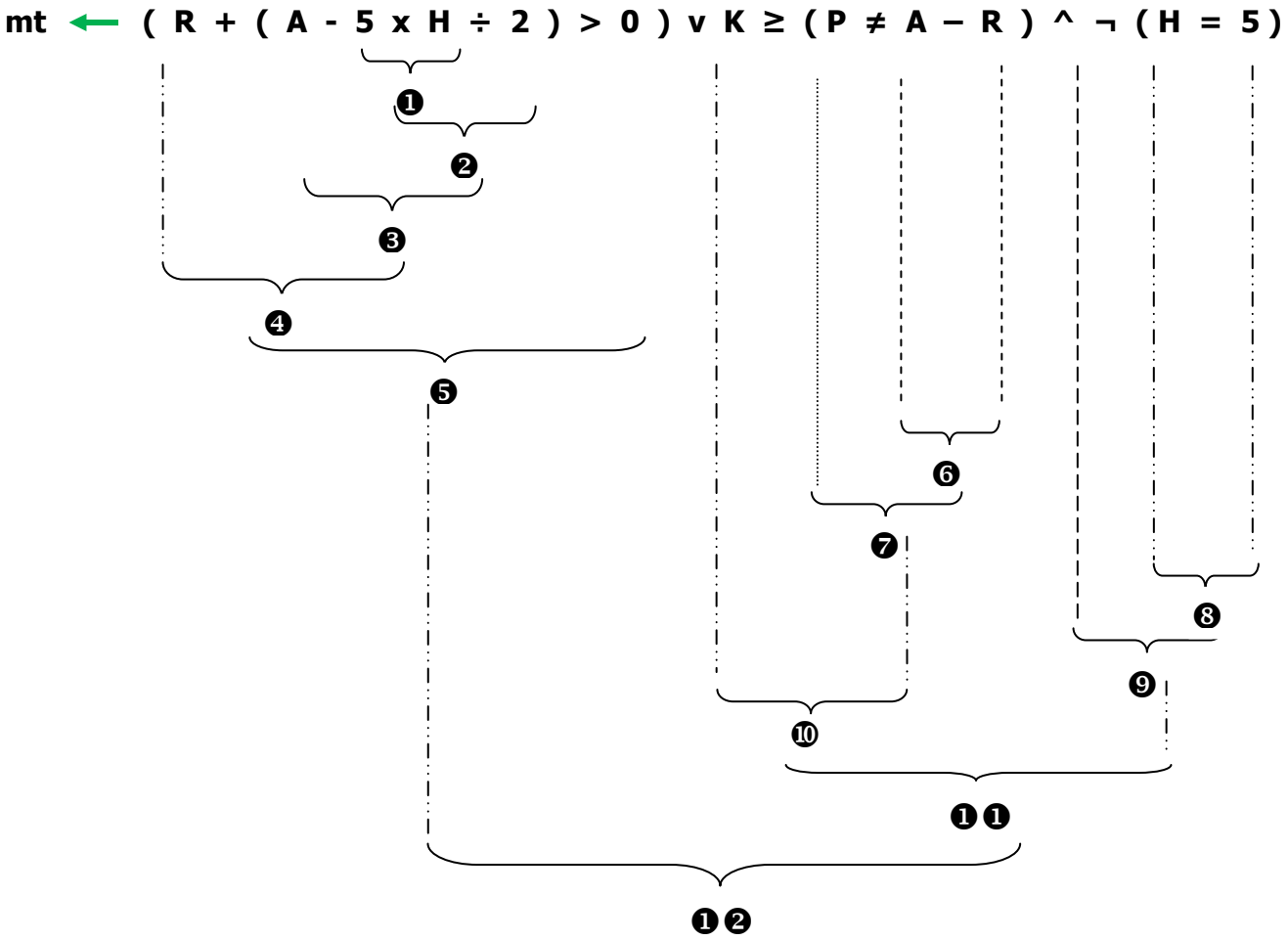
.- luego la operación suma
100 + (-2) resultado parcial $_6$ = 98

.- finalmente la operación resta
98 – 0,75 resultado parcial $_7$ = 97,25

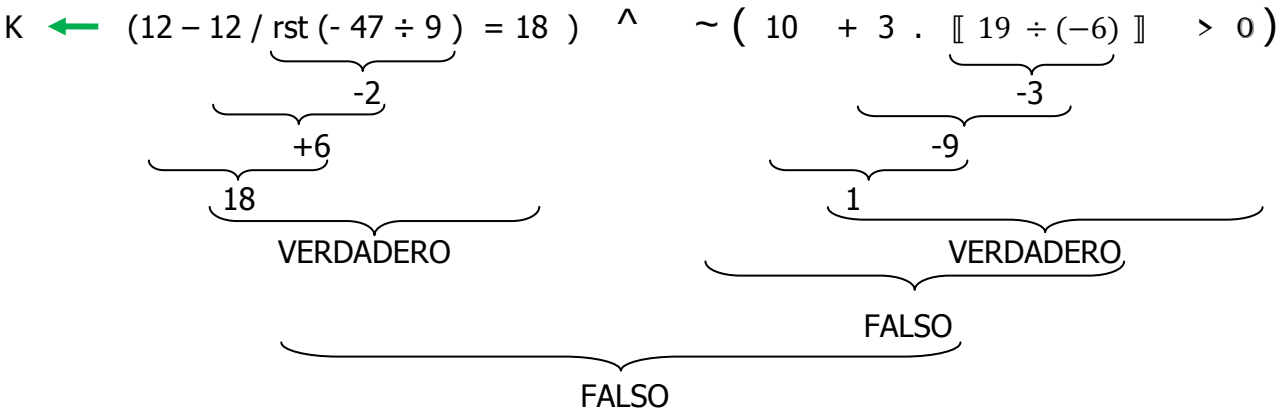
Resultado : Se asigna a la variable Total, el valor **97,25**

Ejercicios sobre el orden de Jerarquía de Operadores

1.- Indicar el orden en que se ejecutarían las operaciones en la siguiente expresión:



2.- Determinar el valor asignado a la variable de la izquierda (K), en la siguiente expresión.



Valor asignado a la variable K : FALSO

5.- Para mostrar o dar salida a resultados (Información de salida)

formato: **ESCRIBIR** **Texto(s), Valor(es), Variable(es), Expresion(es)**

Ejemplos:

Suponiendo que las variables **a, b, edad, nota, precio y clave** contienen los valores siguientes:

a ← 45

b ← 100

edad ← 18

nota ← 18

precio ← 205,75

clave ← VERDADERO

Indicar lo que se muestra en cada una de las siguientes sentencias de salida:

		Se muestra como salida
ESCRIBIR	" Codigo Aprobado"	Codigo Aprobado
ESCRIBIR	"Nota Final: ", nota, "puntos"	Nota Final : 18 puntos
ESCRIBIR	" Precio = ", precio , "IVA =" , 0,12 **.** precio	Precio = 205,75 IVA = 24,69
ESCRIBIR	a , " + ", b , " = " , a + b	45 + 100 = 145
ESCRIBIR	clave, " es diferente a ", b < a	VERDADERO es diferente a FALSO

6.- Para realizar comparaciones de valores, verificar y/o tomar decisiones.

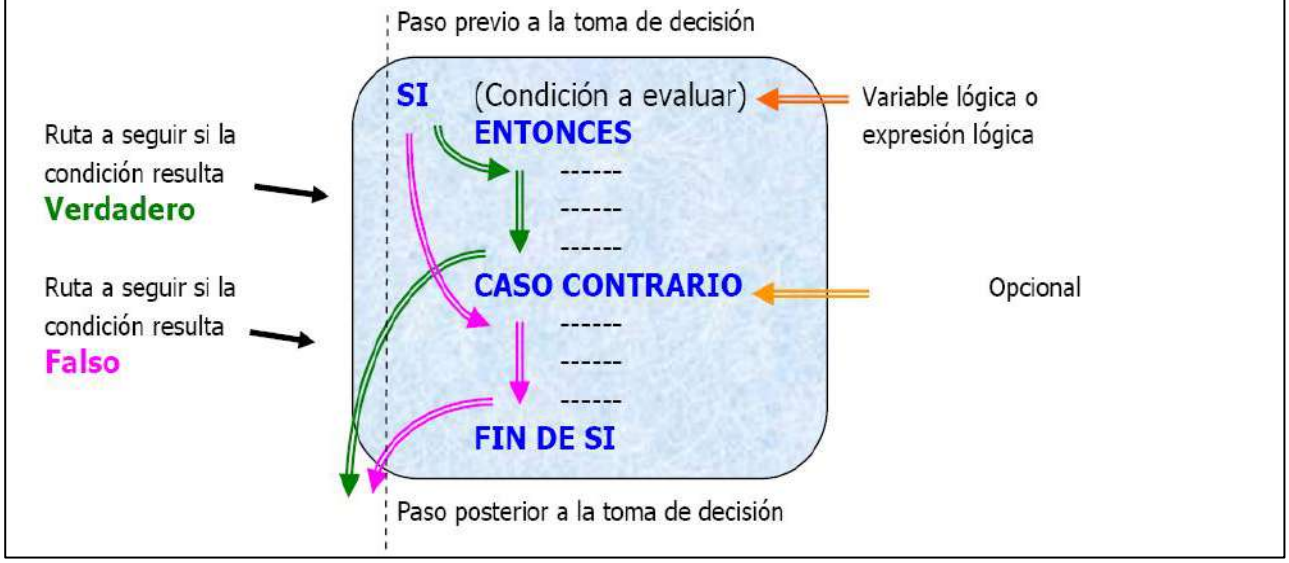


La condición o comparación debe ser una expresión que de cómo resultado un valor lógico (Verdadero o Falso).

¿Cómo funciona?

El SI, evalúa la condición, en caso de resultar VERDADERO, el control del algoritmo se dirige a ejecutar los pasos que se encuentran en el ENTONCES, obvia lo que se encuentre en el CASO CONTRARIO y luego continúa posterior al FIN DEL SI; de resultar FALSO, el control del algoritmo se dirige a ejecutar los pasos que se encuentran en el CASO CONTRARIO, y luego se continúa posterior al FIN DEL SI.

(Nota: En vez de CASO CONTRARIO puede usarse SINO)



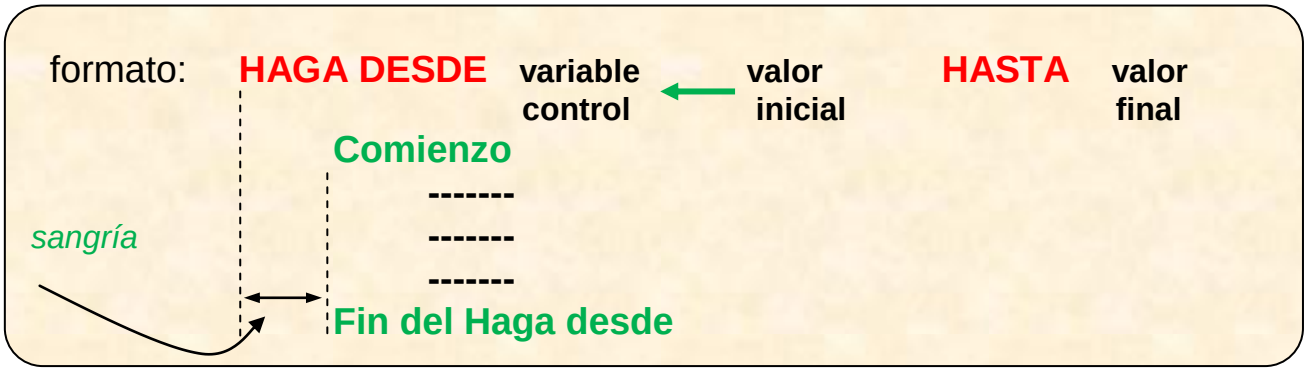
Ejemplo:

```
ESCRIBIR "Ingrese temperatura del proceso, en grados centigrados"
LEER temp
SI   temp ≥ 100
     ENTONCES
        ESCRIBIR " Peligro, la temperatura excede el limite permitido"
     CASO CONTRARIO
        ESCRIBIR " Situación Normal, temperatura dentro del limite permitido"
     FIN DE SI
```

7.- Para repetir un conjunto de instrucciones.
(Estructuras de Repetición, Ciclos o Bucles).

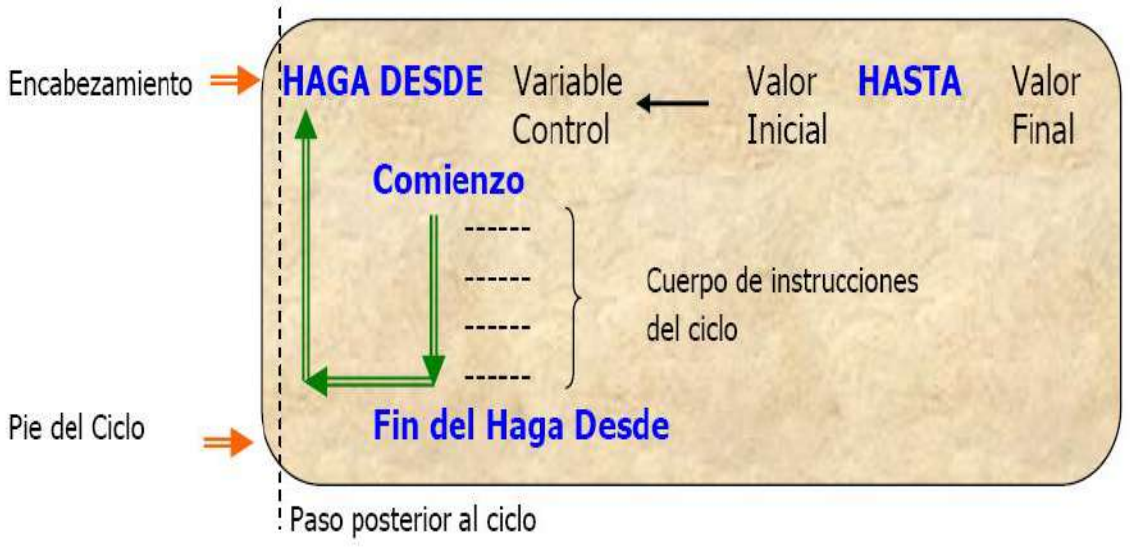
Tipos de Ciclos: { Automático o Determinado ; Condicionales

Ciclo Automático o Determinado:



¿Cómo funciona?

Al entrar por primera vez al encabezamiento del ciclo, a la variable control se la asigna el valor inicial indicado, se compara dicho valor con el valor final, si éste es menor o igual, el control del algoritmo entra a ejecutar todos los pasos o etapas que se encuentren entre el "comienzo" y el "Fin del Haga Desde"; luego se retorna al encabezamiento, la variable control se incrementa en +1, y se vuelve a comparar con el valor final, para entrar de nuevo o no a ejecutar el cuerpo del ciclo. Solo cuando el valor de la variable control no sea menor o igual al valor final, la ejecución del cuerpo del ciclo se detiene y el control se dirige a ejecutar lo que se encuentre luego del "Fin del Haga Desde".



Ejemplo: **HAGA DESDE p ← 1 HASTA 5**

Comienzo

ESCRIBIR " número = " , p

ESCRIBIR " su cuadrado = " , p^2

ESCRIBIR " su cubo = " , p^3

Fin del Haga desde

Al entrar al ciclo, a la variable p se le asigna el valor 1, (p=1), se compara 1 ≤ 5, el resultado es verdadero, por lo tanto se ejecutan las sentencias ESCRIBIR, que muestran lo siguiente:

número = 1
su cuadrado = 1
su cubo = 1

al llegar al Fin del Haga Desde, se retorna al encabezamiento, la variable p se incrementa en +1, es decir su nuevo valor es 2, (p=2), se compara 2 ≤ 5, el resultado es verdadero, por lo tanto se ejecutan, de nuevo, las sentencias ESCRIBIR, que muestran lo siguiente

número = 2
su cuadrado = 4
su cubo = 8

al llegar al Fin del Haga Desde, se retorna al encabezamiento, la variable p se incrementa en +1, es decir su nuevo valor es 3, (p=3), se compara 3 ≤ 5, el resultado es verdadero, por lo tanto se ejecutan, de nuevo, las sentencias ESCRIBIR, que muestran lo siguiente

número = 3
su cuadrado = 9
su cubo = 27

asi sucesivamente, hasta que la variable p toma el valor es 5, (p=5), se compara 5 ≤ 5, el resultado es verdadero, por lo tanto se ejecutan, de nuevo, las sentencias ESCRIBIR, que muestran lo siguiente

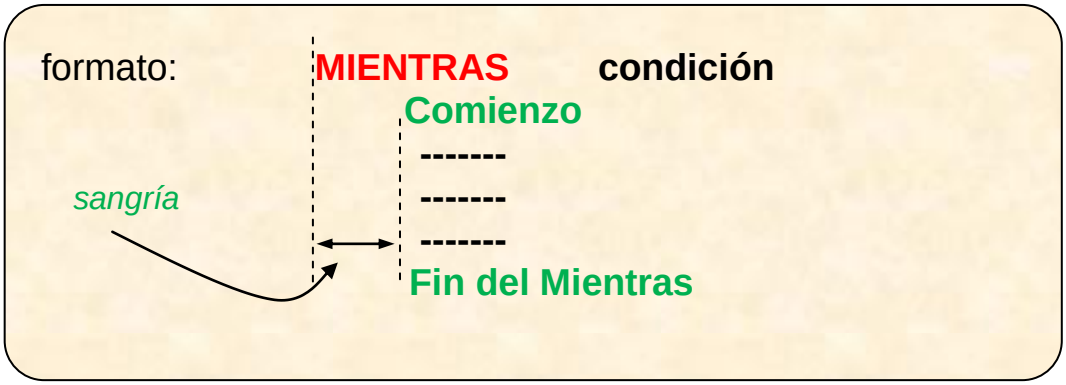
número = 5
su cuadrado = 25
su cubo = 125

al llegar al Fin del Haga Desde, se retorna al encabezamiento, la variable p se incrementa en +1, es decir su nuevo valor es 6, (p=6), se compara 6 ≤ 5, el resultado es falso, por lo tanto el ciclo deja de ejecutarse y el control del algoritmo continua en el paso siguiente al pie del ciclo.

Ciclo Condicional: Existen dos tipos.

Tipos de Ciclos Condicional: { Mientras / Repita-Hasta

Ciclo Condicional Mientras:



¿Cómo funciona?

Al entrar por primera vez al encabezamiento del ciclo, se evalúa la condición, de resultar verdadera, el control del algoritmo entra a ejecutar todos los pasos o etapas que se encuentren entre el "comienzo" y el "Fin del Mientras"; luego se retorna al encabezamiento, para evaluar de nuevo la condición y de seguir resultando verdadera, se seguirá ejecutando el cuerpo del ciclo. Solo cuando el valor de la condición resulte falsa, se detendrá la ejecución del ciclo y el control se dirige a ejecutar lo que se encuentre luego del "Fin del Mientras". Si la primera vez que se entra al ciclo la condición resulta falsa, el ciclo no se ejecutaría ni siquiera una vez.



Ejemplo:

```
x ← 2
MIENTRAS   x < 11
    Comienzo
      SI   rst ( x  ÷  2 ) = 0
           ENTONCES
                ESCRIBIR " El valor de x =  " , x , " es PAR  "
           CASO CONTRARIO
                ESCRIBIR " El valor de x =  " , x , " es IMPAR  "
           FIN DE SI
      x ← x + 1
    Fin del Mientras
```

Se asigna 2 a la variable x (x=2), al entrar al encabezamiento del ciclo MIENTRAS, se compara el valor de x con 11 (2 < 11), como resulta verdadero, el control del algoritmo entra a ejecutar el **SI**, la condición determina el resto de la división 2 ÷ 2, que es cero y al compararlo con cero (0 = 0), resulta verdadero, luego se entra al ENTONCES para generar la salida:

El valor de x = 2 es PAR

se obvia el CASO CONTRARIO, y luego del FIN DE SI, se asigna a x la suma de su valor, +1 (2 + 1 = 3), el Fin del Mientras hace que se retorne al encabezamiento MIENTRAS para evaluar de nuevo (3 < 11), que resulta verdadero, el control del algoritmo entra a ejecutar el **SI**, la condición determina el resto de la división 3 ÷ 2, que es 1 y al compararlo con cero (1 = 0), resulta falso, por lo que se entra al CASO CONTRARIO para generar la salida:

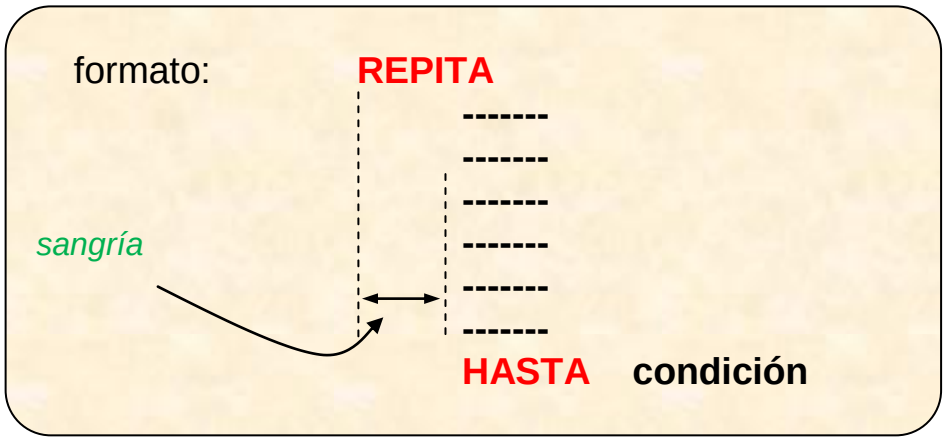
El valor de x = 3 es IMPAR

el proceso continua sucesivamente, generando las siguientes salidas:

El valor de x = 4 es PAR
El valor de x = 5 es IMPAR
El valor de x = 6 es PAR
El valor de x = 7 es IMPAR
El valor de x = 8 es PAR
El valor de x = 9 es IMPAR
El valor de x = 10 es PAR

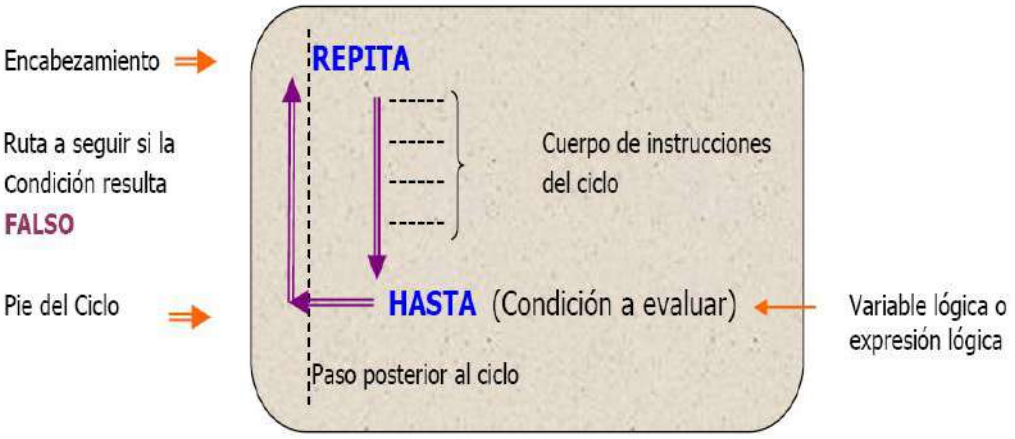
hasta que x toma el valor 11, y a efectuar la compararcion 11 < 11, la condicion del MIENTRAS resulta falso, lo que hace que el ciclo finalice y el control del algoritmo se dirige a la sentencia que se encuentre luego del Fin del Mientras.

Ciclo Condicional Repita Hasta:



¿Cómo funciona?

Al entrar al encabezamiento del ciclo REPITA, el control del algoritmo pasa, sin ninguna condición, a ejecutar todos los pasos o etapas que se encuentren en el cuerpo hasta llegar al pie del ciclo, el HASTA evalúa la condición, si resulta falso, el control del algoritmo retorna al encabezamiento REPITA y se vuelve a ejecutar todos los pasos o etapas que se encuentren en el cuerpo, hasta llegar al pie del ciclo, el HASTA evalúa de nuevo la condición; si resulta verdadero, se detendrá la ejecución del ciclo y el control se dirige a ejecutar lo que se encuentre luego del pie del ciclo. Conviene señalar que este tipo de ciclo se ejecuta obligatoriamente al menos una vez, dado que la condición se encuentra en el pie, luego de haber ejecutado el cuerpo de pasos o etapas.



Ejemplo:

```
m ← - 5
REPITA
    ESCRIBIR " m = " , m , " , su mitad real es = ", m / 2
    ESCRIBIR "  y su mitad entera es = ",  ⟦ m / 2⟧
    m ← m + 1
  HASTA  m > 0
```

Se asigna – 5 a la variable m (m vale - 5), se entra al ciclo REPITA, se genera las salidas:

m = - 5 , su mitad real es = -2,5
y su mitad entera es = - 2

luego, se asigna a m la suma de su valor, +1 (- 5 + 1 = - 4), el pie del ciclo HASTA evalua la condición comparando a m con cero (- 4 > 0), que resulta Falso, el control del algoritmo retorna al encabezamiento REPITA y se vuelve a ejecutar las salidas, con el nuevo valor de m (- 4):

m = - 4 , su mitad real es = -2,0
y su mitad entera es = - 2

el proceso se repite, generando las salidas:

m = - 3 , su mitad real es = -1,5
y su mitad entera es = - 1

m = - 2 , su mitad real es = -1,0
y su mitad entera es = - 1

m = - 1 , su mitad real es = -0,5
y su mitad entera es = 0

m = 0 , su mitad real es = 0
y su mitad entera es = 0

luego de ésta última salida el valor asignado a m, es el valor 1 (0 + 1), el HASTA evalúa la condición (1 > 0), que resulta Verdadero, y se detendrá la ejecución del ciclo, para continuar en la instrucción que se encuentre luego del pie del ciclo.

8.- Para contar eventos, situaciones o casos.

Concepto de Contador:

Concepto de **Contador**: Variable que permite contar las veces que un evento sucede y que atiende a la siguiente expresión:

Variable Contador ⟵ Variable Contador ± Valor Fijo

Ejemplos:	L ⟵	L + 1	Incrementa a L de uno en uno
	M ⟵	M + 2	Incrementa a M de dos en dos
	ZZ ⟵	ZZ + 5	Incrementa a ZZ de cinco en cinco
	P ⟵	P - 1	Decrementa a P de uno en uno
	W ⟵	W - 10	Decrementa a W de diez en diez

9.- Para sumar o acumular valores.

Concepto de Acumulador:

Concepto de **Acumulador**: Variable que permite aglomerar un valor que puede variar y que atiende a la siguiente expresión:

Variable Acumulador ⟵ Variable Acumulador ± Valor

Ejemplos:

Total ⟵ Total + Compra	la variable Total aumenta en el valor de la variable Compra.
Nomina ⟵ Nomina + Sueldo	Incrementa la variable Nomina en el valor de la variable a Sueldo.
R2 ⟵ R2 + Presión	Acumula en R2 el valor de la variable Presión
SAS ⟵ SAS - Gasto	Disminuye el valor de la variable SAS en el valor de la variable Gasto.
H ⟵ H - R1	Decrementa a H en el valor de la variable R1

Observación: Todo contador o acumulador debe inicializarse con un valor previo, generalmente con cero, a través de una asignación, tal como se indica a continuación:

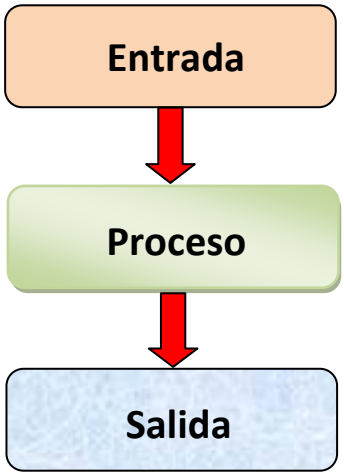
L ⟵ 0 Nomina ⟵ 0

ESTRATEGIAS A SEGUIR EN EL DISEÑO DE ALGORITMOS:

1.- Aplicar la metodología recomendada:

A. Leer el enunciado del problema, tantas veces como sea necesario, hasta comprenderlo muy bien.
B. Analizar, determinando de que datos se disponen, y cuáles deben ser los resultados. Asignar nombre a las variables a definir y describir brevemente su contenido.
C. Planificar, determinando paso a paso, la secuencia de etapas o instrucciones a seguir; se recomienda realizar un bosquejo o esquema previo a escribir el ALGORITMO.
D. Probar, el algoritmo, mediante la prueba de escritorio o la corrida en frio.

2.- En la planificación, aplicar la solución inicial general:



Solicitar datos

Planear la solución

Mostar Resultados

3.-Determinar de acuerdo a las preguntas, el uso o no de estructuras de repetición, simple o anidadas.

4.- Usar y/o aplicar todos los conocimientos previos adquiridos en la formación académica formal.